Role labels and repeated calls in large language model code generation: a task-paired experimental study

Daniil Privezentsev[1*]

1 National Research University Higher School of Economics, 20 Myasnitskaya Ulitsa, Moscow 101000, Russia

* Corresponding author: daprivezentsev@edu.hse.ru

Abstract

Named roles and repeated model calls are often combined in language model workflows, making their separate contributions difficult to assess. We crossed neutral versus planner, implementer and reviewer labels with one versus three calls while holding operation sentences and supplied context fixed. A direct solver provided a fifth condition. The primary model, GPT-5.6 Luna, received 15,000 assignments across 1,000 BigCodeBench programming tasks and three fresh requests per condition. Generation completed for 14,961 assignments; 985 tasks passed reference and incorrect-program controls. Analysis used task-paired per-generation success, retaining repeats within tasks. Role labels changed success by -0.75 percentage points in one call and $+1.35$ points in three calls; neither null hypothesis was rejected after Holm correction, which does not establish equivalence. Three calls reduced success by 4.17 points under neutral labels and 2.13 points under role labels. On complete resource pairs, their API-equivalent valuations were 2.67 and 2.78 times those of the corresponding single-call workflows. The negative call-structure estimates persisted in source-dependence and newly sampled-task analyses. An earlier study with a related model showed the same contrast directions on 200 reused task identifiers but provided no independent task replication. A separate published-controller comparison was limited by substantial infrastructure missingness and did not support a full-population quality ranking. Within the tested full-context workflows, repeated calls increased resource use and lowered executable success on observed eligible pairs. The design does not establish a general effect of independent agents, equal-compute systems or repository-level development.

Introduction

Large language models can generate programs from written specifications, but more elaborate prompting can also use additional computation without improving correctness. Separating the effects of instruction labels from those of repeated calls helps evaluate which components of a workflow warrant their resource use. Here, correctness means that a generated program passes the original benchmark tests, rather than that a model describes its answer as correct.

Code-generation workflows increasingly assign planning, implementation, and review to named LLM roles. Systems such as Self-Collaboration, ChatDev, MetaGPT, AgentCoder, and PairCoder combine role specialization with interaction, generated tests, tool use, or repeated refinement [1–5]. End-to-end comparisons cannot isolate whether reported gains arise from the role names, the operations, additional calls, feedback, or extra computation. That distinction matters in software engineering: each added call can retransmit a large repository context, expose an unverified intermediate response, and increase cost. The primary experiment evaluates sequential prompts sent to one model. Multi-agent systems motivate the design, but the experiment does not test independent agents.

We study a deliberately narrow workflow. The model is asked to analyze requirements and edge cases, implement a solution based on that analysis, and review and revise the program. Executable success is measured by executing the original benchmark tests in a pinned offline environment; the model's textual assertion is never an outcome. We cross two factors while keeping operation sentences and supplied context fixed: neutral versus role labels, and one versus three fresh calls. A direct one-call solver serves as the simpler baseline.

The study asks three research questions. **RQ1:** Do role labels change executable success when the requested operations and call structure are held fixed? **RQ2:** Does splitting those operations across three full-context calls change success and measured resources relative to one call? **RQ3:** How do the staged workflows compare with a direct solver and with a published analyst–developer–tester controller? The first two questions form the prespecified factorial analysis. Comparisons with the direct baseline, the role-by-topology interaction, and subgroup checks are labeled exploratory. The external-controller study has its own prespecified hypothesis-testing family and missingness analysis.

The labels `planner`, `implementer`, and `reviewer` denote prompt segments; they do not imply distinct underlying agents, separate memories, or separate optimization processes. The experiment uses 1,000 BigCodeBench tasks with three fresh requests per condition, and the analysis includes all 14,961 completed candidates. The expanded sample contains 200 task IDs from the earlier study and 800 newly sampled IDs; all responses were regenerated. We report the 200/800 sensitivity separately and do not

call the full sample an independent 1,000-task replication.

We use a crossed design to separate lexical role labels from call topology and report task-paired quality and resource estimates with missingness bounds and a source-dependence audit specified during generation. We then compare the primary results with GPT-5.4-mini, a direct solver, and an adapted Self-Collaboration controller. The direct and interaction comparisons are exploratory, and the study does not claim a general multi-agent effect. In this full-context setting, role labels do not yield a reliable quality benefit, while three-call execution costs substantially more and succeeds less often than the matched one-call workflow.

Related work and scope

Self-Collaboration decomposes code generation into analyst, developer, and tester roles, reports role and interaction-count ablations, and measures token cost [1]. ChatDev and MetaGPT coordinate specialized roles across broader software-development processes [2, 3]. AgentCoder closes a generated-test feedback loop among programmer, test designer, and executor; PairCoder at ASE couples navigator and driver roles with multi-plan exploration and execution feedback [4, 5]. MapCoder combines retrieval, planning, coding, and debugging roles [6]. AdaCoder examines the generalizability of four frameworks across open-source models and introduces adaptive planning and feedback [7, 8]; a distinct ACL paper also named PairCoder evaluates a lighter two-agent scheme across 13 models and explicitly measures tokens [9]. The publicly available six-model AdaCoder preprint already reports that SCC can reduce success while increasing token use [7]; that negative quality–resource pattern alone is therefore not our novelty. Our contribution isolates unchanged-operation label and call-structure interventions. A systematic review identifies 71 primary studies of LLM multi-agent systems for software engineering and highlights the unresolved need to isolate agent capabilities from collaboration effects [10].

Recent controlled comparisons examine homogeneous workflows reproduced by one model, adaptive topology selection, and quality–cost trade-offs [11–13]. These studies vary complete architectures, allocate different compute, or use executable feedback. We instead vary lexical role labels and call boundaries while holding operation sentences

and supplied context fixed. Self-Collaboration is the closest prior workflow. It already compares role-playing instructions with a zero-shot version that removes role-playing and slightly rewrites the operation sentences for naturalness. Our study holds those sentences unchanged and crosses lexical labels with one versus three calls, using repeated task-paired outcomes, full-context resource accounting, and explicit treatment of missing outcomes. Our separate SCC comparison adapts its analyst–developer–tester controller to the same model, tasks, and external evaluator and documents the changes required for this setting. INoT motivates compact internal dialogue and role-like reasoning instructions [14]; we exclude it from the primary factorial family because it changes the reasoning procedure.

Table 1 contrasts the interventions used in related workflow studies.

Table 1. Closest code-generation workflow studies. Execution feedback means runtime test or trace outcomes returned during generation. Resource reporting includes token or call accounting. SCC feedback varies by configuration: its published compiler variant and our adapted author controller execute checks. SCC also studies role removal; our factorial intervention keeps operation sentences unchanged.

Work	Primary variation	Execution feedback	Resources	Direct baseline
SCC [1]	Role configurations and turns	Varies	Yes	Yes
MapCoder [6]	Four-role repair pipeline	Yes	Yes	Yes
AdaCoder [8]	Adaptive planning across models	Yes	Yes	Yes
PairCoder [9]	Two-agent switching across models	No	Yes	Yes
OneFlow [11]	Multi- versus single-model execution	Varies	Yes	Yes
DATS [12]	Five task-adaptive topologies	Varies	Yes	Yes
This work	Labels × one/three calls	No	Yes	Yes

Persona prompting studies likewise distinguish a named persona from a demonstrated performance benefit [15, 16]. Self-Refine and Reflexion use feedback and revision [17, 18]. Execution-guided methods such as CodeT and LDB use generated tests or runtime traces to select or repair programs [19, 20]. Feedback and revision can improve code generation. Execution-guided variants also bundle workflow structure with executable feedback. Our primary matrix withholds all executable feedback until generation ends, so it estimates the effects of labels and call boundaries rather than those of execution-guided repair.

Comparisons involving additional calls can confound call count with computation, repeated context, and exposure of intermediate responses. Matched-thinking-budget work emphasizes that these are distinct interventions [21]. Cost-aware agent evaluation likewise argues for joint accuracy–cost reporting and strong simple baselines [13]. Our topology contrast includes repeated context, additional generation, and exposure of

intermediate responses. We report each resource component and did not impose a post hoc output budget. Agentless and SWE-agent add localization, repository access, or tool interaction [22, 23]; those systems address a broader repository setting than the fixed-context function-generation setting studied here. BigCodeBench supplies diverse library-dependent tasks with executable tests [24]. Native evaluation improves auditability while remaining subject to the benchmark's task–test limitations.

Materials and methods

Workflow and estimands

An engineering task is $(x, \mathcal{C}_0, \mathcal{V})$, where x is the specification, $\mathcal{C}_0$ is the complete supplied context, and $\mathcal{V}$ is an external verification procedure. A protocol a produces a candidate program y and a verification record. In the primary study, $r = 1, 2, 3$ indexes the three requests for task t. When its outcome is available, $Y_{tr}(a) \in \{0, 1\}$ records whether the candidate passes the original tests in a valid environment. Missing outcomes remain undefined. For tasks with all three outcomes, $\bar{Y}_t(a) = \frac{1}{3}\sum_{r=1}^{3} Y_{tr}(a)$ is the per-generation success mean. Each primary contrast averages differences between these means over eligible tasks with all six required outcomes. Thus its estimand is conditional on a complete pair, with equal task weights. We report quality and resources separately because no externally justified joint utility or quality-loss margin is available.

In the single-call core, one model call receives x, $\mathcal{C}_0$, and three ordered operations: analyze requirements and edge cases; implement using the plan; review and revise against the requirements. It returns exactly one fenced Python program. The hidden reasoning trace is unavailable and is not treated as an observed plan or critique score. In the multi-call version, the same three operations are assigned to three fresh calls. Call j receives the task, unchanged context, and every complete preceding response. The final response supplies the candidate. No benchmark test, reference solution, or evaluator feedback is returned to the model.

The five conditions are D (direct implementation), SN (neutral stage labels in one call), SR (role labels in one call), MN (neutral labels across three calls), and MR (role labels across three calls). All structured conditions use identical operation sentences and

final-output contract. The label intervention replaces stage names with `planner`, `implementer`, and `reviewer`. The topology contrast includes repeated full-context transmission and intermediate-response exposure; it is not an effect of call count alone.

The observable state after call j is $\sigma_j = (x, \mathcal{C}_0, a, j, \mathcal{H}_j)$, where $\mathcal{H}_j$ contains complete preceding responses. Generation runs sequentially until the assigned number of calls is complete, or a call is incomplete. Resource availability is checked separately from candidate quality. Incomplete assignments remain unavailable and are not retried automatically. A completed response with zero or multiple recognized Python fences yields an empty observed candidate and a format-error flag. The evaluator then records pass, failure, or unavailable. Only external test execution determines $Y_{tr}(a)$.

Let L be the token length of the task and context, H_j the instruction overhead, E_j the exposed answer length, and O_j all output tokens of call j. Hidden reasoning may contribute to O_j but is not forwarded in E_j. In an additive block model, a three-call sequence uses

$$T_3 = 3L + \sum_{j=1}^{3}(H_j + O_j) + 2E_1 + E_2.$$

A single call uses $T_1 = L + H_s + O_s$. Thus repeated context contributes $2L$ to the difference, while overhead, exposed history, and output lengths account for the remaining terms. The net token difference, $T_3 - T_1$, need not be positive. Because tokenizer boundaries are nonadditive, reported resource values use provider counters. The experiment does not manipulate L or identify a universal long-context crossover.

The compared workflows are summarized in Fig 1.

Fig 1. The crossed intervention. The direct baseline D has one call without staged instructions. Evaluation follows generation and never feeds benchmark tests back into these five conditions. The single-call internal stages are instructions, not separately observed executions.

prompt construction

The common execution instruction is: "Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available." The three exact operation sentences are:

1. Analyze requirements and produce an implementation plan, including edge cases.
2. Implement the requested program or repository change using the plan.
3. Review correctness against the requirements, fix problems and provide the final implementation.

Each sentence is preceded by a label and a colon: either stage 1, stage 2, stage 3 or planner, implementer, reviewer. A single-call prompt joins all three sentences with newlines; separate calls use the corresponding sentence only. D uses "Implement the requested program." All final calls add: "Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available."

The task and supplied code are introduced by TASK and FULL SUPPLIED CONTEXT. Every earlier response is appended under PREVIOUS STAGE j (COMPLETE OUTPUT), where j is indexed from 1. Thus later calls see all earlier text without truncation. Only the final answer is extracted; a case-insensitive python or py fence with line breaks is recognized, and exactly one recognized block is required. Leading and trailing whitespace is removed from the extracted block. Other language tags are ignored. Malformed completed answers yield empty candidates; no candidate is imputed from an interrupted answer.

Experimental design

factorial intervention and estimands

The main study uses BigCodeBench v0.1.4 in its `instruct full` mode [24]. The source pool contains 1,140 tasks. IDs are sorted as strings and shuffled with Python 3.11 `random.Random(20260908)`; the first 40 form the development set. Tasks /0–/7 were public examples used while checking the extraction and evaluation path, so they were excluded before model generation to avoid evaluating on exposed development material. The initial factorial uses the next 200 IDs, and the following 80 form a separate layout set. The expanded allocation retains those 200 IDs, adds 800 from the remaining permutation, and regenerates every response; development and layout IDs remain

excluded. Three repeat labels correspond to fresh requests, not provider sampling seeds. Every task receives all five conditions. The design supplies 15,000 assignments and represents at most 1,000 sampled task units, with each task contributing repeated observations.

For each task, the supplied model input contains the original instruction and code context, including starter code. Tests and canonical solutions are withheld. The final output contract requires exactly one fenced Python block. Before any Luna benchmark responses were generated, a protocol amendment replaced an unsuccessful GPT-5.4-mini run, which produced no candidate responses. The Luna study generated every assignment afresh. All requests used GPT-5.6 Luna at the medium reasoning setting through Codex CLI 0.153.4 [25, 26], with a 600-second per-call deadline and tools, browsing, delegation, and test execution disabled. Inputs above 65,536 UTF-8 bytes are rejected rather than shortened. We do not claim a temperature setting or provider sampling seed.

The primary estimands are four paired task-level differences in three-repeat means:

$$\Delta_{R,1} = \mathbb{E}_t[\bar{Y}_t(\mathrm{SR}) - \bar{Y}_t(\mathrm{SN})], \qquad \Delta_{R,3} = \mathbb{E}_t[\bar{Y}_t(\mathrm{MR}) - \bar{Y}_t(\mathrm{MN})], \tag{1}$$

$$\Delta_{C,N} = \mathbb{E}_t[\bar{Y}_t(\mathrm{MN}) - \bar{Y}_t(\mathrm{SN})], \qquad \Delta_{C,R} = \mathbb{E}_t[\bar{Y}_t(\mathrm{MR}) - \bar{Y}_t(\mathrm{SR})]. \tag{2}$$

For each contrast, all three repeat outcomes in both conditions must be available; the difference of the two three-repeat means then receives equal task weight. The four two-sided one-sample Student's t-tests on these complete-pair differences form one Holm family at $\alpha = 0.05$. We obtain descriptive bootstrap intervals by resampling whole tasks 10,000 times with PCG64 seed 20260909. We do not interpret failure to reject as equivalence or non-inferiority.

Two retrospective diagnostics were added on 3 October 2026 without changing the prespecified tests or their decisions. Four 98.75% Student's t intervals use the Bonferroni construction for approximate simultaneous 95% coverage under the task-level assumptions. An execution-phase check contrasts the first completed generation session with later sessions. Within each phase, only matching observed repeats are paired, their differences are averaged within task, and tasks receive equal weights; 10,000 whole-task PCG64 draws use seed 20261003. Phase task sets overlap and have different repeat

coverage. These diagnostics do not establish equivalence, independence of phases, or absence of provider drift.

The paired design is appropriate because all conditions are evaluated on the same tasks under the same three repeat indices. A task enters a contrast only when its three outcomes are available in both conditions, so the comparison uses paired per-generation means rather than marginal rates with different task mixes. The four contrasts have different eligible task sets, whose sizes are reported with the estimates. Pairing does not eliminate dependence among related benchmark tasks or provider-level correlation across requests.

The t procedures use a large-sample approximation for the mean of bounded task-level differences; they do not require a Gaussian model for individual binary repeat outcomes. Exact finite-sample t calibration is not asserted for the discrete differences. The approximation depends on sufficiently independent task units and a stable outcome-generating process. The recorded analyses do not use a normality-based test-selection rule. Whole-task bootstrap and the specified source-component resampling provide sensitivity checks under different dependence assumptions; neither establishes independence or rules out provider drift.

The source-dependence audit was specified on 11 September while primary generation was in progress, after the initial protocol was fixed; it reads only benchmark sources, not generated candidates or model outcomes. Its union groups tasks connected by exact whitespace-normalized instruction matches, exact Python AST matches after removing leading docstrings, instruction five-word-shingle Jaccard similarity of at least 0.70, or near-code token similarity. A near-code edge requires at least 20 identifier occurrences in each reference program, identifier/literal set Jaccard of at least 0.80, and multiset Jaccard of at least 0.70. Connected components are formed over all 1,140 source tasks and then intersected with the 1,000 allocated tasks. Separate instruction-only partitions use thresholds of 0.50, 0.70, and 0.90. Whole components are resampled, retaining task weights, with PCG64 seed 20260911 and 10,000 draws. This checks a limited form of benchmark dependence [27, 28]; it cannot detect training contamination. The union partition has 992 groups among the 1,000 tasks. Its four quality intervals are reported beside the corresponding contrasts in Table 3.

For descriptive missingness bounds, each unavailable binary endpoint is assigned

once to the unfavorable extreme and once to the favorable extreme, preserving the assigned denominator. The resulting interval answers how far the observed rate or paired difference could move under the recorded missing rows. It is separate from the task bootstrap, which represents sampling variation under the observed eligible outcomes. Neither calculation supplies a causal adjustment for transport failures, deadlines, or task difficulty.

The assignment schedule was fixed before generation, and controls were frozen before model performance was inspected. Prespecification is documented in retained protocol files and local Git chronology; no externally timestamped preregistration was used. Python randomization with order seed 20260908 permuted the five conditions within each task-repeat block and the block order. Actual completion order also depends on concurrency and interruptions; this does not control provider drift. Workers process disjoint task shards with bounded concurrency, but worker order does not determine the condition or task assignment. A continuation amendment could start only never-submitted cells after interruptions; a cell with any started call remained permanently excluded from continuation. This preserves the distinction between the protocol's planned population and the rows that became observed and prevents a failed response from triggering outcome-dependent replacement.

native controls and missingness

Reference controls concatenate the supplied code context and canonical solution. Incorrect controls append a function returning `None`. A task is quality-eligible only when the reference passes and the incorrect control fails. Native evaluation runs in an offline container with two CPUs, a 3-GB memory cap, and the pinned BigCodeBench dependencies. For eligible tasks, a native pass is coded as one, and a native failure or test timeout is coded as zero. Missing generation, an incomplete evaluator run, and an invalid control remain unavailable. All 15,000 assigned rows, including the 15 control-ineligible tasks and incomplete candidates, remain in completion and resource accounting.

Generation produced 14,961 complete candidates; 39 submitted assignments remained incomplete. Every complete candidate entered native evaluation. The reference gate leaves 985 tasks eligible for quality analysis, while 264 assigned endpoints

are unavailable because of generation or control status. Unknown endpoints are not imputed as failures or successes. Assignment-denominator bounds are reported where useful to show the extreme effect of missing binary outcomes; they are not confidence intervals and do not correct selection bias.

The retained endpoint taxonomy separates generation from test failure. Across D, SR, SN, MR, and MN respectively, there are 4, 3, 2, 13, and 17 incomplete generations and 9, 6, 9, 7, and 7 native timeouts. No completed primary response fails the frozen one-block extraction rule. Ordinary native failures account for the remaining unsuccessful evaluated programs. These records identify where failure occurred, but the design does not expose a counterfactual revision trace, so it cannot attribute a three-call failure to a particular intermediate message.

The control gate confirms that the canonical solution runs and that a deliberately incorrect candidate fails before model outcomes are interpreted. A task that fails either control remains in the assignment and resource summaries but is excluded from the quality estimand. This prevents an unavailable or insensitive test from being reclassified after seeing responses. Extraction is equally explicit: a completed answer must contain exactly one recognized fenced Python block. A missing, multiply fenced, or differently tagged answer yields an empty observed candidate and a format flag, while a call that never completes remains unavailable. Thus malformed responses contribute measured generation cost, whereas absent responses may have only partial or unknown counters.

The evaluator runs each candidate against the unchanged original tests in the pinned local environment. Model-call and evaluator timeouts are recorded separately; only the final candidate is tested. Native pass, ordinary failure, and native timeout are retained as distinct records. A model-generated explanation, printed example, or internal review cannot replace an executable pass.

use of AI systems in the research process

The evaluated programs are AI-generated data, so model use is part of the method rather than writing assistance. GPT-5.6 Luna at medium reasoning was assigned 15,000 primary generations during 8–11 September 2026 (UTC). Terminal records for the 6,000 selected SCC-comparison assignments span 11–13 September (UTC). Both studies used Codex CLI/runtime 0.153.4. Of these, 14,961 primary generations and 5,595

SCC-comparison workflows completed; incomplete attempts remain in the assignment ledgers. GPT-5.4-mini generated the earlier 200-task corroborating study on 8 September 2026 [29]. The frozen records retain the exact prompts, model aliases, request order, context bytes, timestamps, status events, extraction outcomes, token counters, and evaluator commands. Provider sampling seeds and immutable weight identifiers were unavailable; no unsupported value is inferred for them. Models had no tools, browsing, repository access, benchmark tests, reference solutions, or evaluator feedback during the primary generation.

OpenAI Codex with GPT-6 assisted with protocol drafting, experiment implementation, analysis and packaging code, consistency checks, literature discovery, and language revision on 8–20 September 2026 and during the final audits on 30 September and 2–3 October. The service was accessed through the desktop and command-line interfaces and did not expose an immutable weight revision. It produced draft protocol prose and portions of the executable code. Validation included unit and integration tests, replay of the numerical analyses from frozen ledgers, comparison of table values with source summaries, checks of cited claims against primary records, and visual inspection of the rendered PDF. These checks were carried out in the AI-assisted workflow; they do not establish that each change received independent human review. Reported numerical results are derived from retained machine-readable records by versioned scripts and checked by the supplied replay procedures. Model prose is not treated as evidence. The frozen protocols and amendments record the design decisions and their timing. The author remains responsible for the submission. AI-assisted reviews served as internal quality control and are not represented as external peer review. Journal-specific restructuring and language revision on 21 October 2026 also used OpenAI Codex; an immutable model revision was not available for this editorial stage. No new experimental generations or candidate executions were performed for this adaptation. Numerical claims and retained table inputs were checked against the frozen source version. This assistance does not constitute independent human peer review.

Results

quality in the 1,000-task study

The observed success rates range from 50.07% to 54.59% across the five conditions. The role-minus-neutral contrast is -0.75 percentage points in one call (95% interval $[-1.97, 0.44]$) and $+1.35$ points in three calls (95% interval $[0.17, 2.56]$). The descriptive interval for the three-call contrast excludes zero, but its Holm-adjusted $p = 0.0599$ exceeds the prespecified familywise threshold. Neither role-label null hypothesis is rejected in the primary analysis.

The call-structure contrasts are larger and point in the same direction. On complete eligible task pairs, three calls reduce success by 4.17 points under neutral instructions and by 2.13 points under role-labeled instructions; both tests reject their null hypotheses after Holm adjustment. Both full-assignment bounds remain below zero even when missing endpoints are assigned values most favorable to the three-call condition. The descriptive role-by-call interaction is $+2.08$ points with interval $[0.42, 3.74]$ on the common 963-task subset; this exploratory analysis was not included in the four-test family. These are per-generation outcomes, not best-of-three selection results.

Table 2 distinguishes assignments, completed generations and observed quality endpoints.

Table 2. Luna factorial study: assignment and observation counts.

Condition	Assigned	Generated	Observed	Passes	Success (%)
D	3000	2996	2951	1611	54.59
SN	3000	2998	2953	1604	54.32
SR	3000	2997	2952	1581	53.56
MN	3000	2983	2938	1471	50.07
MR	3000	2987	2942	1514	51.46

Observed means eligible native quality endpoints, not distinct tasks. Success divides passes by observed repeat outcomes. Each condition has three assigned repeats per task.

Table 3 reports the four primary task-paired contrasts and their missing-outcome bounds.

Table 3. Luna factorial quality contrasts. Differences and bounds are percentage points.

Contrast	Tasks	Difference [95% interval]	Holm p	Assigned bounds
SR – SN	982	-0.75 $[-1.97, 0.44]$	0.2357	$[-2.33, 0.83]$
MR – MN	964	$+1.35$ $[0.17, 2.56]$	0.0599	$[-0.63, 3.37]$
MN – SN	968	-4.17 $[-5.65, -2.75]$	1.34×10^{-7}	$[-6.00, -2.37]$
MR – SR	971	-2.13 $[-3.43, -0.82]$	0.0044	$[-3.83, -0.30]$

Tasks require three observed repeats in each compared condition. Assigned bounds use all 1,000 tasks and every assigned repeat; they are not confidence intervals. Source-group intervals for the four rows are $[-1.99, 0.47]$, $[0.14, 2.60]$, $[-5.69, -2.72]$, and $[-3.44, -0.79]$.

retrospective precision and execution-phase checks

The simultaneous precision intervals are $[-2.32, 0.83]$ points for SR–SN, $[-0.20, 2.90]$ for MR–MN, $[-6.04, -2.29]$ for MN–SN, and $[-3.80, -0.46]$ for MR–SR. The upper compatible label benefit is smaller for one call (0.83 points) than for three calls (2.90 points), although the interval widths are similar. Small label benefits remain compatible with the observations; "no rejected label null" does not mean that both effects are zero. Both topology intervals remain negative under this more conservative interval construction.

The first completed session contains 5,551 candidates; later sessions contain 9,410. Table 4 reports all four phase contrasts. The topology estimates remain negative in both phases, although the early MR–SR interval includes zero. These overlapping, available-case subsets are diagnostic views of the same experiment, not independent replications or a test of time-invariant model weights.

Table 4 separates diagnostic estimates by execution phase.

Table 4. Retrospective execution-phase quality differences (percentage points). Intervals are descriptive task-bootstrap 95% intervals.

Contrast	Phase	Tasks	Repeat pairs	Difference [95% interval]
SR–SN	first	740	1096	-1.42 $[-3.54, +0.74]$
SR–SN	later	936	1855	-0.66 $[-2.23, +0.91]$
MR–MN	first	736	1090	$+0.79$ $[-1.40, +2.99]$
MR–MN	later	934	1844	$+1.41$ $[-0.11, +2.93]$
MN–SN	first	737	1092	-4.27 $[-6.56, -1.99]$
MN–SN	later	934	1846	-4.46 $[-6.37, -2.61]$
MR–SR	first	738	1092	-2.06 $[-4.27, +0.14]$
MR–SR	later	934	1849	-2.37 $[-4.00, -0.77]$

A task contributes the mean of its observed matching repeats in that phase. Task sets overlap; these estimates are not independent replications and do not change the primary Holm family.

direct baseline and sampling-history audits

The estimated direct-solver differences from the one-call workflows are small: D–SN is +0.27 points and D–SR is +0.95 points, with both intervals crossing zero (Table 5). These comparisons do not establish equivalence. On observed task pairs, D's success rate is 3.20–4.59 percentage points higher than that of the three-call workflows. Because the direct contrasts were not part of the four prespecified factorial tests, we report them as exploratory estimates rather than extending the confirmatory family after observing results.

Table 5 reports exploratory estimates against the direct solver.

Table 5. Exploratory direct-solver quality contrasts. Differences and intervals are percentage points; positive values favor D.

Contrast	Tasks	Difference	95% interval
D − SN	980	+0.27	[−0.88, 1.43]
D − SR	979	+0.95	[−0.17, 2.11]
D − MN	966	+4.59	[2.97, 6.18]
D − MR	970	+3.20	[1.82, 4.64]

Intervals resample tasks. These post hoc contrasts are outside the four-test Holm family and support estimation, not a new confirmatory family.

The topology estimates are also negative in both sampling-history subsets (Table 6). The neutral three-call effect is −4.17 points in both the 200 reused task IDs and the 800 newly sampled IDs. The role-labeled topology contrast is negative in both subsets, with greater uncertainty in the smaller reused subset. Role-label contrasts remain small and uncertain. This analysis cannot rule out benchmark contamination, but it shows that the principal topology result is not driven only by the previously studied 200 task IDs.

Table 6 compares reused and newly sampled task identifiers.

Table 6. Old/new task sensitivity for the four primary contrasts. Values are percentage points with task-bootstrap 95% intervals.

Contrast	Subset	Tasks	Difference [95% interval]
SR–SN	reused 200	194	−1.03 [−3.78, 1.72]
	new 800	788	−0.68 [−2.07, 0.72]
MR–MN	reused 200	190	+1.93 [−0.70, 4.56]
	new 800	774	+1.21 [−0.13, 2.58]
MN–SN	reused 200	192	−4.17 [−7.64, −0.87]
	new 800	776	−4.17 [−5.84, −2.58]
MR–SR	reused 200	190	−1.23 [−3.68, 1.23]
	new 800	781	−2.35 [−3.88, −0.81]

Every response was generated afresh. "Reused" means that the task ID had appeared in the earlier 200-task study; it is not reused output.

resource accounting

For completed candidates, resource accounting includes the full repeated inputs and exposed intermediate responses. Single-call role-labeled instructions have a valuation ratio of 0.939 relative to neutral instructions, or 6.08% lower API-equivalent valuation, on 997 tasks with all three repetitions observed in both conditions; their token ratio is 0.986. Within three calls, the corresponding valuation ratio is 0.978 and token ratio 0.990 on 979 task pairs. These label-associated resource differences are descriptive and smaller than those observed in the earlier mini study.

Three-call neutral instructions use 3.08 times as many tokens and 2.67 times the valuation of single-call neutral instructions. With roles, the ratios are 3.09 and 2.78. Removing the cache discount leaves valuation ratios of 2.78 and 2.87. The monetary figures are API-equivalent valuations derived from measured counters, not subscription invoices or total research cost. Reasoning tokens are included in output and are not added twice.

For scale, the mean known totals per completed candidate are 3,972 tokens and \$0.000813 for D, 4,073 and \$0.000899 for SR, 4,131 and \$0.000957 for SN, 12,583 and \$0.002496 for MR, and 12,712 and \$0.002552 for MN. Known valuation for completed candidates across the five arms is \$23.07. Known stages of incomplete workflows add \$0.03, bringing the all-known-turn subtotal to \$23.10; 32 attempted stages have unavailable counters and are excluded from these subtotals. Ratios remain the primary resource comparison because small completion-count differences make unpaired totals misleading.

For input count I, cached portion C, and output count O, the Luna valuation is $[0.20(I - C) + 0.02C + 1.20O]/10^6$ dollars using the disclosed September 2026 base reference rates [25, 30]. It values observed counters at a published API rate; it does not estimate provider overhead, researcher time, evaluator compute, cache-write surcharges, or total cost of ownership. The no-cache sensitivity uses $[0.20I + 1.20O]/10^6$ and tests whether the direction depends on discounted cached input. Every transmitted context and exposed preceding response is included, because that exposure is part of the treatment. Unknown counters are reported as unknown and are not zero-imputed.

Table 7 reports ratios on complete resource pairs.

Table 7. Luna factorial resource ratios: first condition divided by second.

Contrast	Tasks	Tokens	Valuation	95% interval	No cache
SR – SN	997	0.986	0.939	[0.93, 0.95]	0.955
MR – MN	979	0.990	0.978	[0.97, 0.99]	0.986
MN – SN	983	3.078	2.672	[2.63, 2.71]	2.777
MR – SR	986	3.090	2.780	[2.74, 2.82]	2.867

Ratios divide sums of three-repeat task means on complete resource pairs. The displayed interval is for the API-equivalent valuation ratio. No cache removes its input-cache discount; subscription billing is not estimated.

corroborating studies

An earlier GPT-5.4-mini factorial study used the same 200 task IDs later retained in the expanded allocation, but generated one fresh request per condition under the earlier model. Of 1,000 assignments, 996 were completed; the contrasts include 191–193 eligible task pairs. All four estimated effects have the same direction as in the Luna study (Table 8): role-label effects are negative with one call and positive with three calls, whereas call-structure effects are negative under both label settings. None of the four tests rejects its null hypothesis after Holm correction. The corroboration concerns direction across two model aliases on the same 200 tasks, not an independent task sample or a second high-powered confirmation. The mini study's three-call valuation ratios are 2.09 against SN and 2.50 against SR.

Table 8 summarizes the earlier-model corroboration.

Table 8. Earlier GPT-5.4-mini corroboration on the 200 reused task IDs. Differences and descriptive intervals are percentage points; n counts complete task pairs. The four exact McNemar tests form a Holm family.

Contrast	n	Δ	95% interval	Holm p
SR–SN	191	−1.05	[−4.71, +2.62]	1.000
MR–MN	193	+1.04	[−3.11, +5.70]	1.000
MN–SN	191	−4.71	[−9.42, 0.00]	.313
MR–SR	193	−2.07	[−6.22, +2.07]	1.000

Other retained pilots vary instruction placement or change the reasoning procedure. We do not pool these pilots with the primary study.

Comparison with the published Self-Collaboration workflow

We compare Self-Collaboration (SCC) with the single-call conditions using the same Luna model, medium reasoning and native evaluator. This comparison concerns complete workflows. The adapted author controller [31] begins with an analyst JSON

plan, calls a developer for Python, and asks a tester to generate up to five example calls in a `check(candidate)` function. It executes the program and generated check; exception-free execution is its internal stopping signal, while printed values are not automatically compared with expected answers. The configured cap permits one initial implementation and at most one repair. The model receives the complete supplied task context and serialized role history. Reference solutions and benchmark tests remain external.

The original schedule contains 3,000 task-repeat blocks, each assigning fresh SR, fresh SN and SCC workflows. An amendment adopted after generation began, before inspecting SCC quality, fixes its first 2,000 randomized blocks (order seed 20260911). This gives 6,000 method assignments over 956 tasks: 212 tasks have one selected repeat, 444 have two, and 300 have three. Of these tasks, 941 pass reference controls. The unselected 3,000 method assignments are outside the amended denominator.

All 5,595 completed programs underwent native evaluation: 1,997 SR, 1,998 SN and 1,600 SCC. The comparison retains 397 infrastructure failures and eight historically paused attempts. Specifically, SCC has 396 Docker failures and four paused workflows; SR has three paused attempts; SN has one paused attempt and one stream failure. No submitted attempt was regenerated. Quality is observed for 1,964 SR repeats, 1,965 SN repeats and 1,574 SCC repeats after reference controls, with 1,067, 1,064 and 809 passes, respectively. These are observed-repeat counts, not independent tasks or best-of-three rates.

For each comparator, we first average SCC-minus-comparator outcome differences within each task over matching selected repeats with both outcomes observed. We then average these task means with equal weight across eligible tasks with at least one matched repeat. Thus a task with three selected repeats has the same weight as a task with one. Two-sided task-level t-tests for SCC–SR and SCC–SN form a separate two-test Holm family. Intervals use 10,000 whole-task PCG64 draws, seed 20260911 and linear percentile endpoints.

Table 9 reports the observed-pair controller comparisons.

Table 9. SCC quality on observed matched repeats. Differences and intervals are percentage points; n counts contributing tasks.

Contrast	n	Δ	95% CI	Holm p
SCC–SR	883	-2.68	$[-4.72, -0.60]$	.0217
SCC–SN	882	-2.48	$[-4.50, -0.43]$	.0217

Both conditional contrasts are negative (Table 9). The source-group sensitivity intervals are similar: $[-4.72, -0.58]$ points against SR and $[-4.51, -0.43]$ against SN for the union partition. However, fixed-denominator extreme bounds across all 941 eligible selected tasks are $[-12.98, +6.61]$ and $[-13.14, +6.43]$ points. They allow either sign. The observed-pair results therefore do not establish SCC inferiority over the full assigned population.

Resource comparisons require both complete workflows with valid counters. The ratio of paired task means for API-equivalent valuation is 3.138 against SR (896 tasks; 95% interval $[3.067, 3.208]$) and 2.939 against SN (895 tasks; $[2.871, 3.010]$). These ratio-of-means intervals use 10,000 whole-task draws with seed 20260914. They concern the observed complete workflows and do not combine quality preservation with a monetary advantage. Availability and matched effects before and after the amendment are retained as descriptive checks; they do not identify a causal time effect.

A separate diagnostic evaluates all 399 retained developer programs from incomplete SCC workflows without restarting the controller or appending its generated checks. It finds 196 passes among 392 control-eligible programs; seven programs are control-ineligible. These intermediate-program results do not replace any unknown whole-workflow outcome in the comparison.

Discussion

The completed factorial study estimates a conditional difference between the tested workflows. In the full-context, no-compression protocol, distributing planning, implementation, and critique across three fresh calls increases repeated transmission and measured output and yields lower native-test success on observed task pairs than keeping the same operations in one response. The result is specific to the tested call structure, model, task population, deadline, and evaluation contract. It does not show

that separate agents are generally ineffective or that additional interaction cannot help when agents have tools, retrieval, repository state, or a matched computational budget.

The role-label intervention is smaller and uncertain. The three-call role estimate is descriptively positive, but it does not meet the Holm-adjusted significance threshold in the primary analysis. The estimates are imprecise and differ in sign across call structures, so they do not establish a benefit from naming prompt stages in this setting. Failure to reject also does not establish quality equivalence. Internal labels can still organize a prompt or communicate a useful decomposition; the experiment measures whether they changed this benchmark outcome under identical operation sentences.

Resource use should be interpreted alongside quality. Single-call role labels slightly reduce measured valuation in this study, while three calls cost roughly 2.7–2.8 times as much. Because the design does not impose a strict matched output budget, the accounting includes the induced reasoning length, repeated context, intermediate responses, and cache state. No joint quality-cost superiority claim is made. The no-cache sensitivity changes the magnitude but preserves the large topology contrast. On the complete three-repeat resource pairs, MN–SN adds 8,582 provider-observed tokens per generation and MR–SR adds 8,510. Both paired valuation differences are approximately $1.60 per 1,000 generations at the disclosed rates. The absolute monetary increment is small for these function tasks; latency and total deployment cost were not measured.

The external evaluator also prevents a workflow from receiving credit for a persuasive self-reported correction that did not change executable behavior. Only the final candidate enters the native tests. The direct condition has the lowest observed resource means. Its estimated quality differences from the one-call staged conditions are small and their intervals include zero; these exploratory tests were not designed to establish equivalence. It has higher success on observed task pairs than both three-call conditions, although those comparisons were not prespecified as a confirmatory family.

In this setting, direct and one-call staged workflows are the lower-cost baselines. Additional calls warrant evaluation when they add parallel search, private state, tools, retrieval, or executable feedback. Evaluations should report both task-paired executable quality and provider-observed token counters, with the model and input, output, and cache accounting specified; these counts depend on provider tokenization.

Three repeats improve the estimate for sampled tasks but do not turn the 15,000 assignments into independent task units. The bootstrap keeps repeats together, and each task contributes an equal-weighted per-generation mean. This avoids treating repeated requests as new task units or using best-of-three selection.

The auxiliary studies vary one dimension at a time: model, instruction placement, or reasoning procedure. Their results remain separate from the Luna quality tests and do not resolve benchmark training contamination.

Limitations and reproducibility

The work uses two model aliases, each at one reasoning setting and one client version, and does not identify model-family effects. The smaller mini study reuses the same 200 task IDs and has only one request per condition; its directional agreement is corroboration rather than independent replication. The provider did not expose an immutable weight identifier, a temperature setting, or sampling seeds. The expanded sample reuses those 200 task IDs, so it is not 1,000 newly held-out tasks. Public benchmark availability leaves training contamination unresolved, and related tasks may reduce effective independence.

Quality remains unknown for unavailable generations and infrastructure failures. Observed test failures and benchmark timeouts count as unsuccessful programs on eligible tasks. After transport interruptions, continuation was allowed only for assignments that had not yet started; started calls were not retried. Available-case estimates concern the observed eligible subset. Extreme assignment bounds show what missing binary endpoints permit but cannot remove non-random missingness. The study therefore avoids claims of equivalence, non-inferiority, or an economically optimal design.

Native tests provide executable evidence but inherit the benchmark's specification-test relationship. Reference and incorrect controls detect unusable environments and insensitive negative checks; they do not prove that every assertion captures the natural-language task. The fixed-context protocol excludes retrieval, tools, repository interaction, and adaptive test feedback. Consequently, the results do not establish performance on repository repair or end-to-end software development.

The intervention is narrower than the broader prototype that motivated it: context

selection, selective reruns, an auxiliary checker, and compression are disabled. This isolates the staged-generation core but cannot substitute for a full deployment evaluation. The SCC comparison has an additional limitation because its generated checks and exception-free stopping signal define a different feedback controller. Its higher valuation and lower success on observed task pairs apply to that workflow as a whole; they do not isolate the contribution of role labels within SCC. Substantial SCC infrastructure missingness limits population-level conclusions.

Task IDs, prompts, responses, statuses, extraction flags, counters, controls, native reports, and analysis seeds are retained in the project records. The retained records allow reconstruction of the assignment denominators and paired subsets. A new provider revision, dependency version, or task-test interpretation could change the rates. Reproduction should preserve the disclosed alias, client, context bytes, deadlines, control gate, and missingness rules and report any deviation.

Conclusions

Additional calls can be a measurable liability. On complete eligible task pairs, separating unchanged full-context operations into three calls reduced executable success by 2.13–4.17 percentage points: approximately 21–42 fewer passes per 1,000 generations at those estimated rates. On complete resource pairs, valuation increased by factors of 2.67–2.78. Both quality assignment bounds remain negative, and the direction persists in the 800 newly sampled task IDs and both execution phases. These results support a concrete engineering baseline: evaluate a direct solver and a single-call staged prompt before paying for a longer pipeline.

Role names are a smaller, unresolved intervention. Neither role-label null is rejected after Holm correction. The retrospective simultaneous intervals allow a gain up to 0.83 points in one call and 2.90 points in three calls on the respective observed subsets; they do not establish equivalence. Naming stages and adding calls must therefore be assessed separately.

The conclusion is about the tested workflow. Direct-solver comparisons are exploratory, and substantial SCC missingness prevents a full-population quality ranking. The design does not isolate intermediate-message exposure from additional reasoning

and repeated context, impose equal compute, or test independent model families or repository development. No claim of a generic multi-agent penalty follows from these results.

Data availability

The frozen replication data and analysis software are publicly accessible in the versioned archive cited here [32]. A byte-identical copy is supplied as S1 Data, file `S1_Data.zip`. Its historical filename is `FSE2027-supplement.zip`; that filename identifies the source artifact and does not imply journal acceptance or a current conference status. The archive includes protocols and amendments, task selections, assignment-level outcomes, status and usage counters, primary candidate programs, all primary native reports, controls, environment specifications, benchmark evaluator source and data under their original license, source-dependence partitions, the adapted Self-Collaboration controller and its upstream license, analysis scripts, exact table inputs and file hashes. These files support numerical reanalysis of the primary, earlier-model, external-controller and source-dependence studies without provider credentials. An archive-local environment recipe is provided, but a fresh build and equivalent rerun of native tests have not been verified. The released data do not include full conversational responses, external-controller candidate source text or operational transport traces; these were not used as numerical analysis inputs and may contain incidental generated identifiers or machine-local metadata. The paper does not claim complete release of those omitted materials or a verified fresh native replication. A release inventory in the submission package identifies these boundaries.

Acknowledgments

The research and editorial uses of artificial intelligence are disclosed in Materials and methods. Artificial intelligence systems are not listed as authors.

Author contributions

Daniil Privezentsev: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing. The research was conducted with the AI assistance disclosed in Materials and methods; these contribution labels do not imply independent human verification of every analysis step.

References

1. Dong Y, Jiang X, Jin Z, Li G. Self-Collaboration Code Generation via ChatGPT. ACM Transactions on Software Engineering and Methodology. 2024;33(7):1-38. Preprint: arXiv:2304.07590. Available from: https://doi.org/10.1145/3672459. doi:10.1145/3672459.

2. Qian C, Liu W, Liu H, Chen N, Dang Y, Li J, et al. ChatDev: Communicative Agents for Software Development. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics; 2024. p. 15174-86. Available from: https://aclanthology.org/2024.acl-long.810/. doi:10.18653/v1/2024.acl-long.810.

3. Hong S, Zhuge M, Chen J, Zheng X, Cheng Y, Zhang C, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In: The Twelfth International Conference on Learning Representations; 2024. Available from: https://openreview.net/forum?id=VtmBAGCN7o.

4. Huang D, Zhang JM, Luck M, Bu Q, Qing Y, Cui H. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation; 2023. Available from: https://arxiv.org/abs/2312.13010. arXiv:2312.13010. doi:10.48550/arXiv.2312.13010.

5. Zhang H, Cheng W, Wu Y, Hu W. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In: Proceedings of the 39th IEEE/ACM International Conference on Automated

Software Engineering. ACM; 2024. p. 1319-31. Available from: https://doi.org/10.1145/3691620.3695506. doi:10.1145/3691620.3695506.

6. Islam MA, Ali ME, Parvez MR. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics; 2024. p. 4912-44. Available from: https://aclanthology.org/2024.acl-long.269/. doi:10.18653/v1/2024.acl-long.269.

7. Zhu Y, Liu C, He X, Ren X, Liu Z, Pan R, et al.. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation; 2025. Version 1, 5 April 2025; six-model preprint, distinct from the subsequent journal publication. Available from: https://arxiv.org/abs/2504.04220v1. arXiv:2504.04220.

8. Zhu Y, Liu C, He X, Ren X, Liu Z, Pan R, et al. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation. IEEE Transactions on Software Engineering. 2026;52(2):631-50. Available from: https://doi.org/10.1109/TSE.2025.3642621. doi:10.1109/TSE.2025.3642621.

9. Chen J, Li X, Xu Y, Cui Y, Weng F, Zhao H, et al. PairCoder: Pair Programming-Inspired Two-Agent Collaboration for Code Generation. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics; 2026. p. 3043-58. Available from: https://aclanthology.org/2026.findings-acl.149/. doi:10.18653/v1/2026.findings-acl.149.

10. He J, Treude C, Lo D. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. ACM Transactions on Software Engineering and Methodology. 2025;34(5):1-30. Available from: https://doi.org/10.1145/3712003. doi:10.1145/3712003.

11. Xu J, Koesdwiady A, Bei S, Han Y, Huang B, Wang D, et al.. Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline; 2026. Available from: https://arxiv.org/abs/2601.12307. arXiv:2601.12307.

12. Hong Y. Learning How Much to Collaborate: Difficulty-Aware Topology Selection for Multi-Agent Code Generation; 2026. Available from: https://arxiv.org/abs/2609.13890. arXiv:2609.13890.

13. Kapoor S, Stroebl B, Siegel ZS, Nadgir N, Narayanan A. AI Agents That Matter. Transactions on Machine Learning Research. 2025. Available from: https://openreview.net/forum?id=Zy4uFzMviZ.

14. Sun H, Zeng S. Introspection of Thought Helps AI Agents; 2025. Available from: https://arxiv.org/abs/2507.08664. arXiv:2507.08664.

15. Zheng M, Pei J, Logeswaran L, Lee M, Jurgens D. When "A Helpful Assistant" Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. In: Findings of the Association for Computational Linguistics: EMNLP 2024. Association for Computational Linguistics; 2024. p. 15126-54. Available from: https://aclanthology.org/2024.findings-emnlp.888/. doi:10.18653/v1/2024.findings-emnlp.888.

16. Luz de Araujo PH, Röttger P, Hovy D, Roth B. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics; 2025. p. 26857-86. Available from: https://aclanthology.org/2025.emnlp-main.1364/. doi:10.18653/v1/2025.emnlp-main.1364.

17. Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegreffe S, et al. Self-Refine: Iterative Refinement with Self-Feedback. In: Advances in Neural Information Processing Systems. vol. 36; 2023. Available from: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html. doi:10.52202/075280-2019.

18. Shinn N, Cassano F, Gopinath A, Narasimhan K, Yao S. Reflexion: Language Agents with Verbal Reinforcement Learning. In: Advances in Neural Information

Processing Systems. vol. 36; 2023. Available from: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html. doi:10.52202/075280-0377.

19. Chen B, Zhang F, Nguyen A, Zan D, Lin Z, Lou JG, et al. CodeT: Code Generation with Generated Tests. In: The Eleventh International Conference on Learning Representations; 2023. Available from: https://openreview.net/forum?id=ktrw68Cmu9c.

20. Zhong L, Wang Z, Shang J. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step. In: Findings of the Association for Computational Linguistics: ACL 2024. Association for Computational Linguistics; 2024. p. 851-70. Available from: https://aclanthology.org/2024.findings-acl.49/. doi:10.18653/v1/2024.findings-acl.49.

21. Tran D, Kiela D. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets; 2026. Available from: https://arxiv.org/abs/2604.02460. arXiv:2604.02460.

22. Xia CS, Deng Y, Dunn S, Zhang L. Demystifying LLM-Based Software Engineering Agents. Proceedings of the ACM on Software Engineering. 2025;2(FSE):801-24. Available from: https://doi.org/10.1145/3715754. doi:10.1145/3715754.

23. Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, et al.. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering; 2024. Published as a NeurIPS 2024 paper; arXiv version retained for stable metadata. Available from: https://arxiv.org/abs/2405.15793. arXiv:2405.15793.

24. Zhuo TY, Vu MC, Chim J, Hu H, Yu W, Widyasari R, et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In: International Conference on Learning Representations; 2025.

Published as a conference paper at ICLR 2025. Available from: https://arxiv.org/abs/2406.15877.

25. OpenAI. GPT-5.6 Luna Model Documentation and Pricing; 2026. Available from: https://developers.openai.com/api/docs/models/gpt-5.6-luna.

26. OpenAI. Codex Non-Interactive Mode and JSONL Event Stream; 2026. Available from: https://developers.openai.com/codex/noninteractive.

27. Allamanis M. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In: Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. ACM; 2019. p. 143-53. Available from: https://doi.org/10.1145/3359591.3359735. doi:10.1145/3359591.3359735.

28. MacKinnon JG, Nielsen MØ, Webb MD. Cluster-Robust Inference: A Guide to Empirical Practice. Journal of Econometrics. 2023;232(2):272-99. Available from: https://doi.org/10.1016/j.jeconom.2022.04.001. doi:10.1016/j.jeconom.2022.04.001.

29. OpenAI. GPT-5.4 Mini Model Documentation; 2026. Available from: https://developers.openai.com/api/docs/models/gpt-5.4-mini.

30. OpenAI. API Pricing; 2026. Standard text-token rates. Available from: https://developers.openai.com/api/docs/pricing.

31. Dong Y. Self-Collaboration Code Generation: Accompanying Python Software; 2024. Historical author implementation associated with arXiv:2304.07590, pinned at commit b471e12051190dbae2c71b429a3c87466df4b336; workflow and transport adaptations are specified in the text. Available from: https://github.com/YihongDong/Self-collaboration-Code-Generation.

32. Privezentsev D. Role labels and call structure in code generation: frozen replication data and analysis software; 2026. Version 008b730a905277079710092ecf3881770eb100fc; archive SHA-256 693c088ac83b9f78ea48f786393cd8b7fddea4a77206a6e6d386fbd2626b9d71; accessed 21 October 2026. Available from: https://github.com/daniil-novel/

`article-INoT/raw/008b730a905277079710092ecf3881770eb100fc/`
`submission/fse2027/upload/FSE2027-supplement.zip`.

Supporting information

S1 Data. Frozen numerical data and analysis software. The archive contains protocols and amendments, task selections, assignment-level outcomes and usage counters, primary generated programs and native evaluation reports, evaluator controls, environment specifications, adapted controller source, analysis software, exact table inputs and file hashes. Its version and public source are identified in Data availability. The archive does not supply omitted conversational bodies or controller candidate source text and does not establish a verified fresh native reconstruction. The archive is supplied as `S1_Data.zip`; its historical source filename was `FSE2027-supplement.zip`.